

Testing Foundations (1/2)

SE 413/513 — Software Testing
Grand Valley State University

Learning Objectives — Lecture 1/2

By the end of today, you should be able to:

- Explain why systematic testing matters, using real-world examples of what happens when it fails.
- Distinguish an **error**, a **fault**, and a **failure**.
- Explain the difference between **verification** and **validation**.
- State and apply the seven fundamental principles of software testing.

Roadmap

- 1 Why Do We Test?
- 2 Error, Fault, and Failure
- 3 Verification and Validation
- 4 The Seven Principles
- 5 Wrap-Up

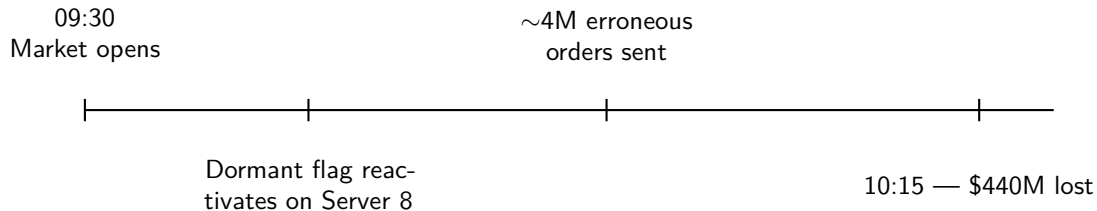
Real-World Case Study

Real-World Case Study

Knight Capital Group (August 1, 2012). A deployment engineer pushed new trading code to 7 of 8 production servers. The 8th server still had a dormant, seven-year-old test module that reused an old order-tag flag.

When markets opened, that flag reactivated the obsolete code on the 8th server, sending roughly 4 million erroneous orders into the market.

Knight Capital: The Timeline



In just 45 minutes, Knight Capital lost approximately \$440 million and nearly went bankrupt.

Key Idea

Key Idea

A defect is **cheap** to fix when it is caught early, and can be **catastrophic** when it is caught late — or not caught at all.

We will return to this idea more precisely — with an actual cost curve — in Lecture 2/2.

What Is Software Testing?

Key Idea

Software testing is the process of evaluating a system to determine whether it meets specified requirements and to identify defects *before users do*.

It is easy to treat testing as a bureaucratic afterthought — something to do once the “real” work of writing code is finished.

Professional practice treats it very differently: testing is one of the primary ways an engineering team **manages risk**.

Discussion Prompt

Discussion Prompt

Think of a software failure you've personally experienced or heard about — an app crash, a website outage, a payment error.

What do you think made it costly (in money, time, trust, or safety)? Could disciplined testing plausibly have caught it?

Roadmap

- 1 Why Do We Test?
- 2 Error, Fault, and Failure**
- 3 Verification and Validation
- 4 The Seven Principles
- 5 Wrap-Up

Three Words We Often Confuse

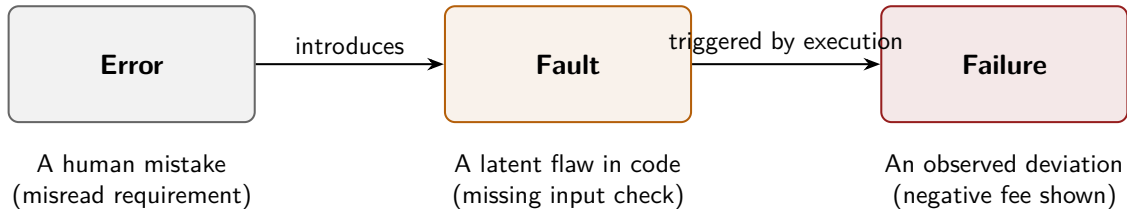
Casual conversation uses “bug” for almost anything that goes wrong. Software engineering distinguishes three related but different concepts.

Error (mistake): a human action that produces an incorrect result — it happens in a person's head, not in the software.

Fault (defect/bug): a flaw in the software that *results from* an error — a static, latent property that can sit unnoticed for a long time.

Failure: an *observable* deviation from expected behavior, occurring when a fault is actually executed under the right conditions.

The Chain: Error → Fault → Failure



Key Idea

A fault with no triggering conditions may never cause a failure at all — which is exactly why testing (deliberately trying to *trigger* faults) is necessary, rather than just waiting for users to report problems.

The Chain in Code: Setting Up the Fault

A developer misunderstands a billing requirement (the *error*): they believe a late fee can never be negative, so they never considered a negative `days_late`.

```
def calculate_late_fee(days_late, daily_rate=1.50, max_fee=25.00):  
    """Return the late fee for an overdue library book."""  
    fee = days_late * daily_rate  
    if fee > max_fee:  
        return max_fee  
    return fee
```

```
>>> calculate_late_fee(3)  
4.5  
>>> calculate_late_fee(30)  
25.0
```

For every ordinary input, the function behaves correctly — the fault is **invisible** here.

The Chain in Code: Exposing the Failure

```
>>> calculate_late_fee(-4)
-6.0
```

A negative fee makes no business sense, but nothing in the code prevents it.

The fault (missing input validation) was present the whole time; it just needed the right **triggering condition** (a negative `days_late`) to produce an observable failure.

Tip

Tip

When you write a defect report, be precise: “The *failure* was a negative late fee shown to the customer; the *fault* was missing input validation in `calculate_late_fee()`; the underlying *error* was an incomplete requirements assumption about how `days_late` could arrive.”

This precision is what lets a team fix the actual fault instead of just patching one symptom.

Discussion Prompt

Discussion Prompt

Identify a *second*, different fault that could be introduced into `calculate_late_fee()` by a plausible requirements misunderstanding.

Describe the fault in code terms, and describe a specific input that would trigger a failure.

Roadmap

- 1 Why Do We Test?
- 2 Error, Fault, and Failure
- 3 Verification and Validation**
- 4 The Seven Principles
- 5 Wrap-Up

Real-World Case Study

Real-World Case Study

Mars Climate Orbiter (1999). NASA's \$327.6 million orbiter was lost when it approached Mars at the wrong altitude and disintegrated. Lockheed Martin's ground software calculated thruster performance in pound-force-seconds (imperial); JPL's navigation software expected newton-seconds (metric), per the interface specification.

Each side's software worked correctly against *its own* assumptions — the failure was an **unverified interface** between two components, not a coding typo.

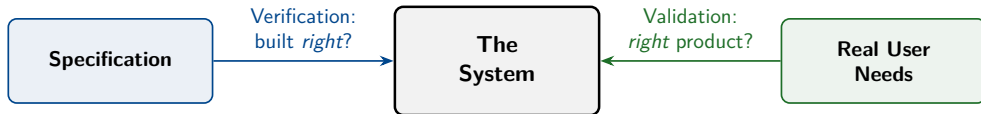
Two Questions That Sound Similar

Confusing these two is one of the most common sources of expensive mistakes in software engineering.

Verification: “Are we building the product *right*?” Does the software conform to its specification?

Validation: “Are we building the *right* product?” Does the software satisfy the actual needs of its users, in the real world it will operate in?

Verification vs. Validation



Key Idea

It is entirely possible to **pass verification and still fail validation**. A component can perfectly implement its specification, and the software can still be wrong — if the specification itself did not capture what was actually needed.

Two Correct Components, One Wrong System

```
# navigation_module.py -- Team A. Expects newton-seconds (metric),  
# per the interface spec both teams agreed to.  
def compute_trajectory_correction(thrust_newton_seconds):  
    return thrust_newton_seconds * 0.0142
```

```
# ground_software.py -- Team B. Their tools use pound-force-seconds  
# (imperial); nobody converted the value before the call.  
from navigation_module import compute_trajectory_correction  
def send_correction(thrust_pound_force_seconds):  
    return compute_trajectory_correction(thrust_pound_force_seconds)
```

Each function is easy to *verify* in isolation — both teams' unit tests pass. The *system* fails validation.

Real-World Case Study

Real-World Case Study

Mars Climate Orbiter (1999). NASA's \$327.6 million orbiter was lost when it approached Mars at the wrong altitude and disintegrated. Lockheed Martin's ground software calculated thruster performance in pound-force-seconds (imperial); JPL's navigation software expected newton-seconds (metric), per the interface specification.

Each side's software worked correctly against *its own* assumptions — the failure was an **unverified interface** between two components, not a coding typo.

Common Pitfall

Common Pitfall

“It passed all its tests” is *not* the same claim as “it is correct.”

A test suite can only verify a system against the specification (or against the tests themselves). If the specification is wrong, incomplete, or ambiguous, a fully *verified* system can still be *validated* as unfit for its real purpose.

Discussion Prompt

Discussion Prompt

A team builds a search feature that returns results in under 200ms, exactly as specified, and every unit test passes. After launch, users complain the results are frequently irrelevant.

Was this a verification failure, a validation failure, or both? Justify your answer.

Roadmap

- 1 Why Do We Test?
- 2 Error, Fault, and Failure
- 3 Verification and Validation
- 4 The Seven Principles**
- 5 Wrap-Up

The Seven Principles: Overview

Widely taught in the testing community, and codified by organizations such as ISTQB. Treat them as a mental checklist whenever you design a test strategy.

1

Shows presence,
not absence

2

Exhaustive testing
impossible

3

Early testing
saves money

4

Defect
clustering

5

Pesticide
paradox

6

Context-
dependent

7

Absence-of-errors
fallacy

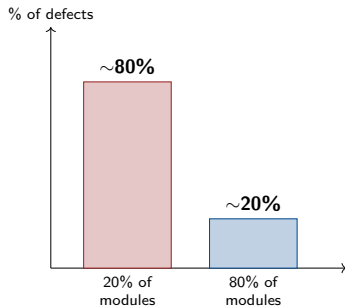
Principles 1 & 2

- 1. Testing shows the presence of defects, not their absence.** Running tests and finding no failures does not prove the software is defect-free — only that, for the inputs you tried, no failure was observed.
- 2. Exhaustive testing is impossible.** A function accepting two 32-bit integers already has over 1.8×10^{19} possible input pairs. Testing must choose a representative, risk-informed subset.

Principles 3 & 4

3. Early testing saves time and money. A misunderstood requirement caught in design review costs a conversation; the same mistake caught after release can cost a re-engineering effort and reputational damage. This is **shift-left testing**.

4. Defect clustering. Most defects concentrate in a small number of modules — a rough Pareto (80/20) pattern.



Principle 5: The Pesticide Paradox

If you run the exact same set of tests repeatedly, they eventually stop finding new defects — just as insects eventually develop resistance to a pesticide used repeatedly.

To keep finding new defects, test cases must be regularly reviewed, revised, and extended.

Principle 6: Testing Is Context-Dependent

There is no single “correct” way to test all software. The testing strategy for a safety-critical medical device is (and must be) very different from a marketing landing page.

Risk tolerance, regulatory requirements, and the cost of failure all shape how much testing is appropriate.

Real-World Case Study

Real-World Case Study

Therac-25 (1985–1987). A radiation therapy machine whose software controlled beam intensity and mode. Unlike its predecessor, it removed *hardware* safety interlocks and relied on software alone.

A race condition let a fast-typing operator switch treatment modes during an 8-second data-entry window in a way the software did not anticipate, bypassing the safety check.

At least six patients received massive overdoses; at least three died.

Principle 7: Absence-of-Errors Is a Fallacy

Even if you find and fix every defect you can locate, that alone does not guarantee success.

If the system does not fulfill user needs, it can still fail — even with **zero known bugs**. This connects directly back to **validation**.

Real-World Case Study

Real-World Case Study

Ariane 5 Flight 501 (1996). 37 seconds after launch, the rocket self-destructed, destroying its \$370+ million payload. A 64-bit float (horizontal velocity) was converted into a 16-bit signed integer, causing an overflow.

The module was reused, unchanged, from Ariane 4, where that value could never exceed the 16-bit range. It was **never re-validated** against Ariane 5's faster trajectory.

The module had “zero known errors” on Ariane 4 — irrelevant to whether it was validated for its new context.

Key Idea

Key Idea

Read together, the seven principles say something quite humbling: testing can build **justified confidence**, but it can never provide **proof of correctness**.

Good testing is about managing risk intelligently, not about reaching some mythical state of “fully tested.”

Discussion Prompt

Discussion Prompt

The Ariane 5 case study is most directly connected to which of the seven principles?

Explain the connection in your own words, and identify a *second* principle that also applies to this case study.

Roadmap

- 1 Why Do We Test?
- 2 Error, Fault, and Failure
- 3 Verification and Validation
- 4 The Seven Principles
- 5 Wrap-Up**

Recap: Key Takeaways

Vocabulary is not pedantic — it is precision. Error, fault, and failure let a team fix the right thing.

Verification and validation ask different questions. Passing every test does not guarantee you built the right product.

Testing manages risk; it does not produce proof. The seven principles keep expectations realistic.

Real incidents are case studies, not trivia. Knight Capital, Mars Climate Orbiter, Therac-25, and Ariane 5 each map directly onto a concept you'll use all course.

Looking Ahead to Lecture 2/2

Today answered *why* we test and *what* testing can and cannot promise.

Next lecture: how defects are classified and tracked (severity vs. priority, the defect life cycle), why cost grows the later a defect is found, the four levels of testing, and the V-model/STLC that organize testing across a whole project.

Questions?

Thank you.

SE 413/513 — Software Testing
Grand Valley State University